SmartClinic System

Multi-Tenant Clinic Management Platform

System Design Document

March 2026

1. Introduction

SmartClinic is a multi-tenant clinic management platform designed to support multiple clinics under a unified system. Each clinic operates as an isolated tenant, sharing the underlying infrastructure while maintaining strict data separation. The system supports five distinct user roles: Super Admin, Clinic Admin, Receptionist, Doctor, and Patient.

This document presents the complete system design including the class diagram, entity-relationship diagram (ERD), data flow diagrams (DFD), use case diagram, activity diagrams, and sequence diagrams.

2. Class Diagram

The class diagram below illustrates the main entities of the SmartClinic system and their relationships. The system centres around a base User class that is extended by Doctor and Receptionist subclasses. Patient is a separate entity linked to the clinic via ClinicId. Appointments link Doctors, Patients, and Clinics together.

Key classes:

• Clinic – stores clinic identity (ClinicId, Name, Address, Phone, Email)

• User – base class with UserId, Name, Email, Password, Role (enum: SuperAdmin, ClinicAdmin, Receptionist, Doctor, Patient), and ClinicId

• Doctor – extends User; adds Specialization

• Receptionist – extends User

• Patient – holds Contact information and ClinicId (for tenant isolation)

• Appointment – links Doctor, Patient, and Clinic with Date, Time fields

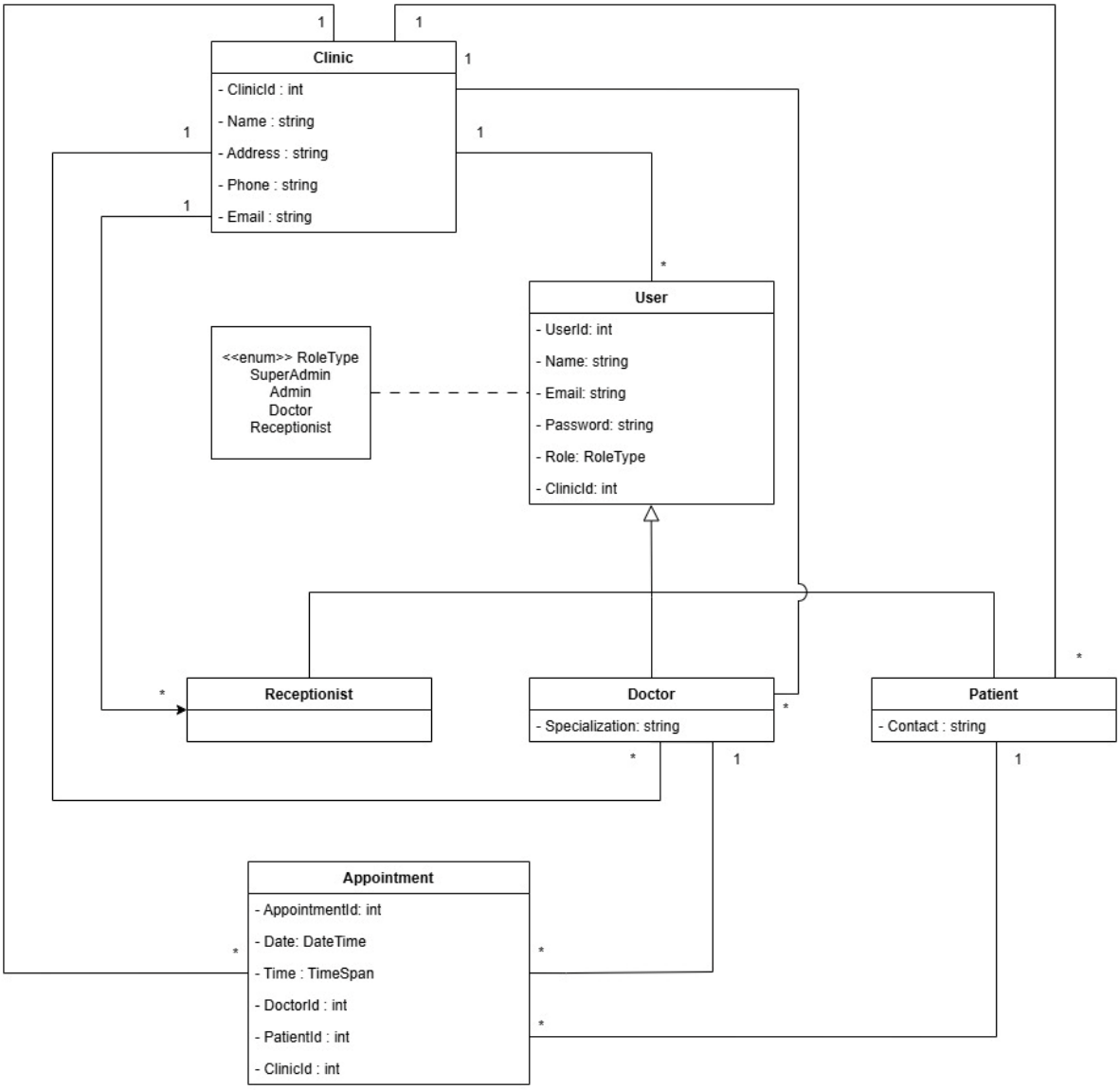


Figure 1 – Class Diagram

3. Entity-Relationship Diagram (ERD)

The ERD represents the database schema for the SmartClinic system. It uses crow's-foot notation to show cardinality between entities. The central entities are clinics, users, patients, doctors, appointments, and roles.

Key relationships:

• A clinic employs many users; a user belongs to one clinic.

• A clinic has many appointments (contains); appointments store DoctorId, PatientId, and ClinicId as foreign keys.

• Each user is assigned one role (via the Roles table); roles define the level of access within the system (SuperAdmin, ClinicAdmin, Receptionist, Doctor, Patient).

• A patient can book many appointments; a doctor attends many appointments.

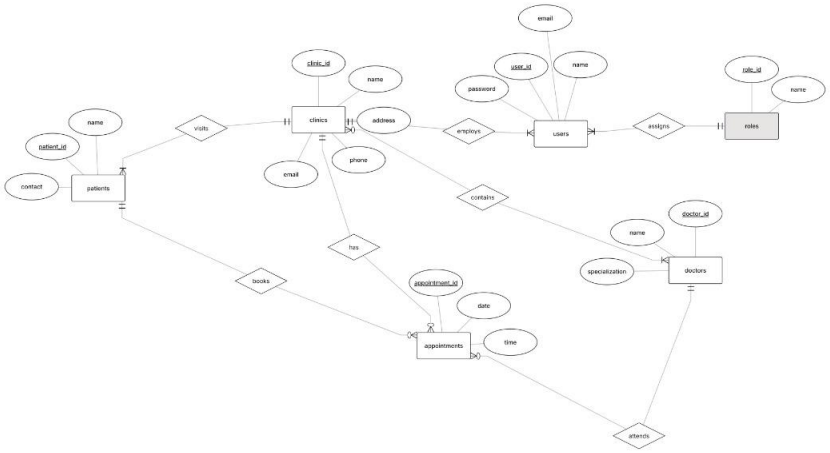


Figure 2 – Entity-Relationship Diagram (ERD)

4. Data Flow Diagrams (DFD)

4.1 DFD Level 0 – Context Diagram

The Level 0 diagram (context diagram) shows the SmartClinic System as a single process interacting with five external entities: Patient, Receptionist, Doctor, Clinic Admin, and Super Admin. Each entity sends requests and receives responses from the central system.

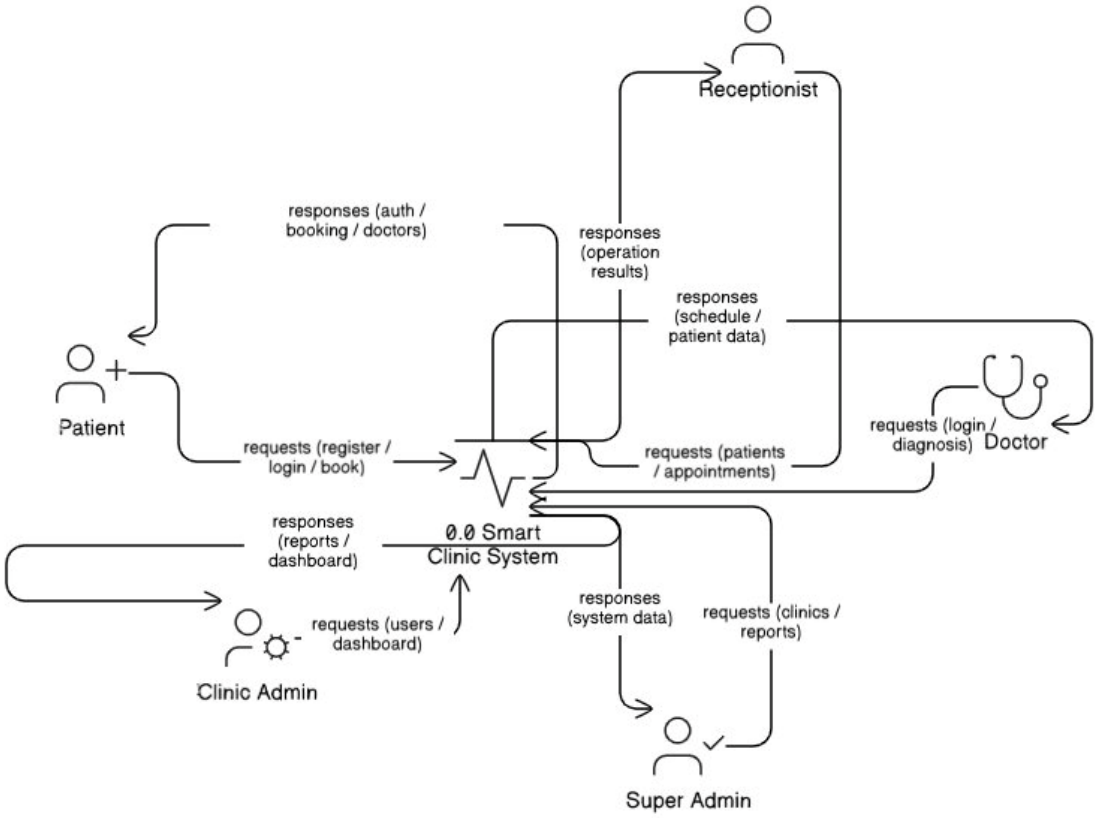


Figure 3 – DFD Level 0 (Context Diagram)

4.2 DFD Level 1 – Decomposed Processes

The Level 1 diagram decomposes the system into nine major processes, each supported by dedicated data stores:

• 1.0 Authentication – validates credentials for all user roles; issues auth tokens

• 2.0 Manage Clinics – allows Super Admin to add/edit/delete clinic records (D1 Clinics DB)

• 3.0 Manage Users – Clinic Admin adds/updates/deletes doctor and receptionist accounts (D2 Users DB)

• 4.0 Manage Patients – receptionist manages patient records (D3 Patients DB)

• 5.0 Manage Appointments – schedule, update, and cancel appointments (D4 Appointments DB)

• 6.0 View Doctors – patients and receptionists browse the doctor directory

• 7.0 Doctor Operations – doctors view patient records and add diagnoses/prescriptions (stored in D3 Patients DB)

• 8.0 View Reports – Super Admin and Clinic Admin access system reports

• 9.0 View Dashboard – role-specific dashboard data for admins

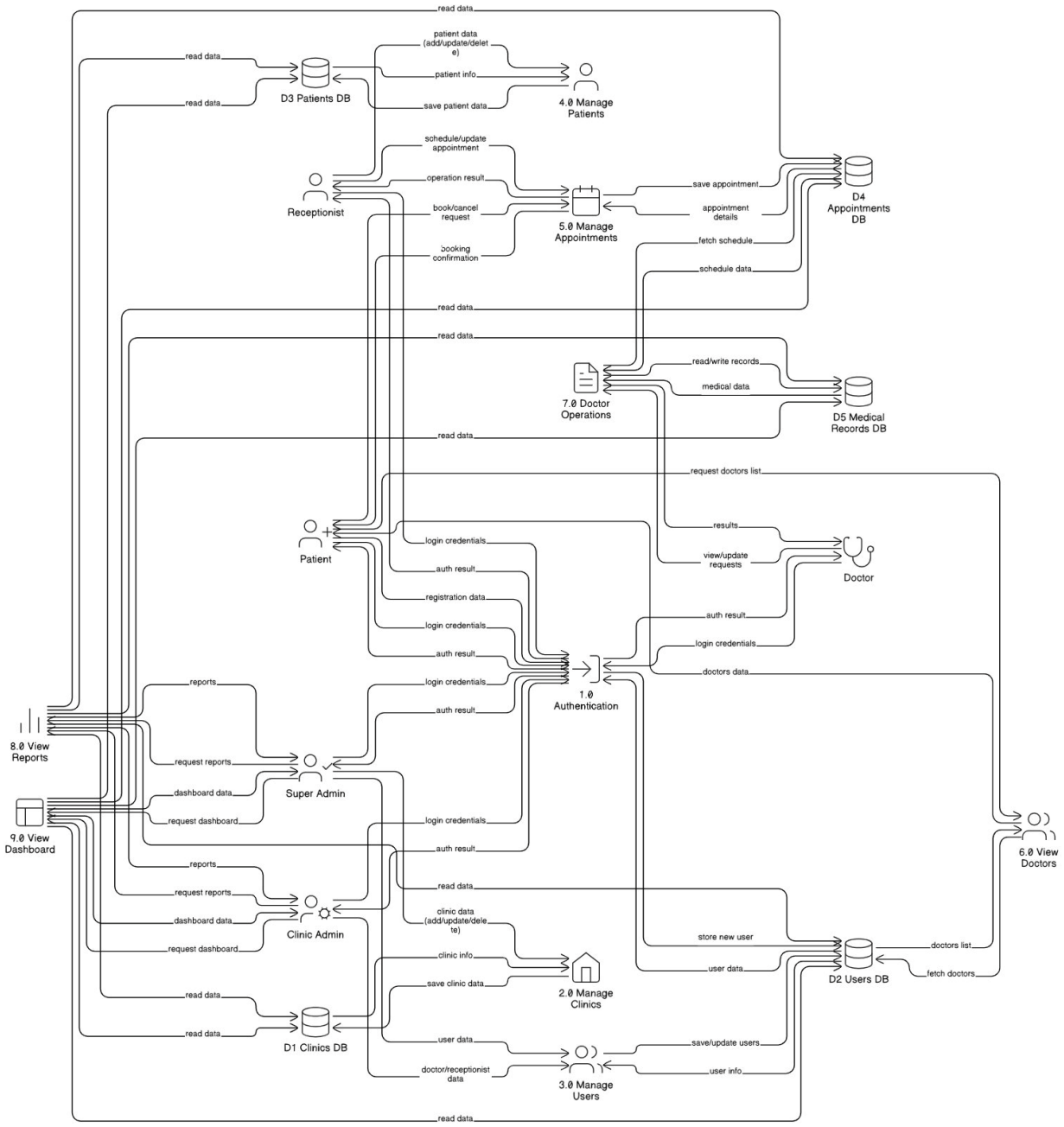


Figure 4 – DFD Level 1

5. Use Case Diagram

The use case diagram shows all interactions between the five actors and the SmartClinic system. All protected use cases include the «Authenticate User» use case. Key use cases per actor are summarised below:

• Super Admin – Login, Logout, Manage Clinics, Manage All Users, View Reports

• Clinic Admin – Login, Logout, Manage Receptionists, Manage Doctors, View Reports, View Dashboard

• Receptionist – Manage Patients, Manage Appointments, Login, Logout

• Patient – Register, Login, Book Appointment, Cancel Appointment (extends Book), View Doctors, View Schedule

• Doctor – Login, Logout, View Schedule, View Patient Records, Add Diagnosis / Prescription

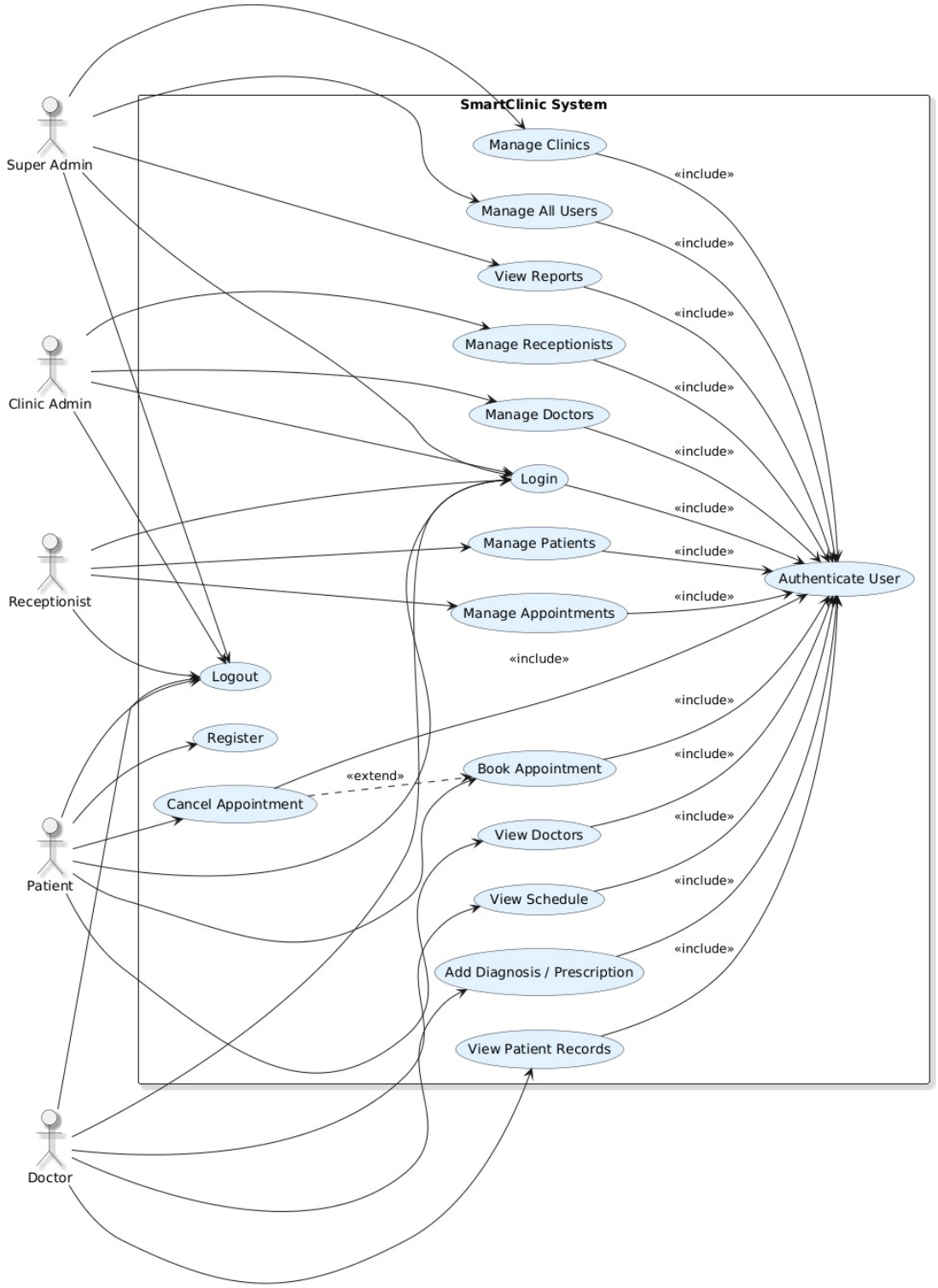


Figure 5 – Use Case Diagram

6. Activity Diagrams

Activity diagrams model the step-by-step workflow for each major use case. Each diagram starts with a login and credential-validation step before proceeding to the core operations.

6.1 Login / Authentication Flow

The user enters their email and password. If credentials are valid, the system reads the user's role and redirects to the appropriate dashboard (Super Admin, Clinic Admin, Receptionist, Doctor, or Patient). Invalid credentials display an error message and return to the login screen.

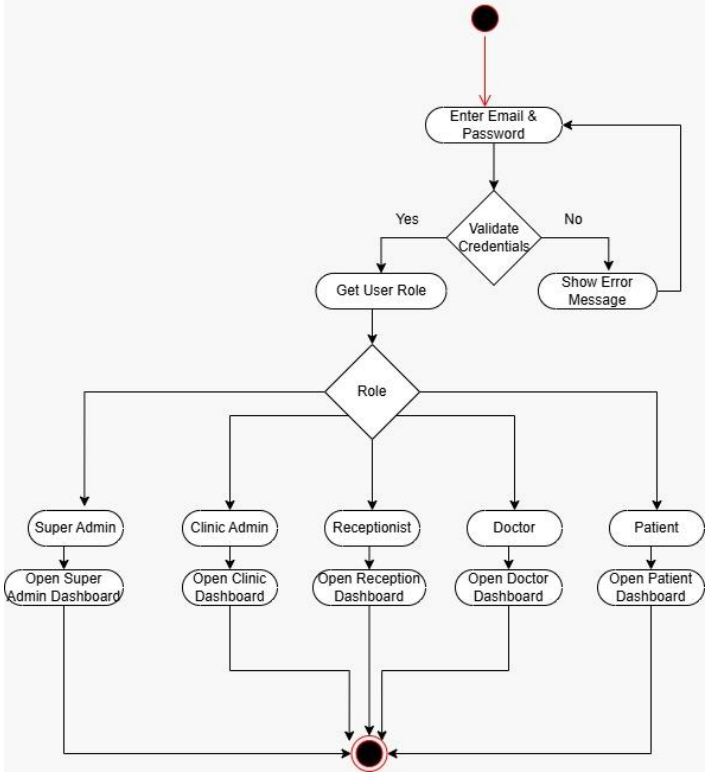


Figure 6 – Activity Diagram: Authentication Flow

6.2 Doctor Workflow

After successful login, the Doctor views their dashboard and appointment list. They select an appointment, view patient details, enter a diagnosis and prescription, and save the medical record.

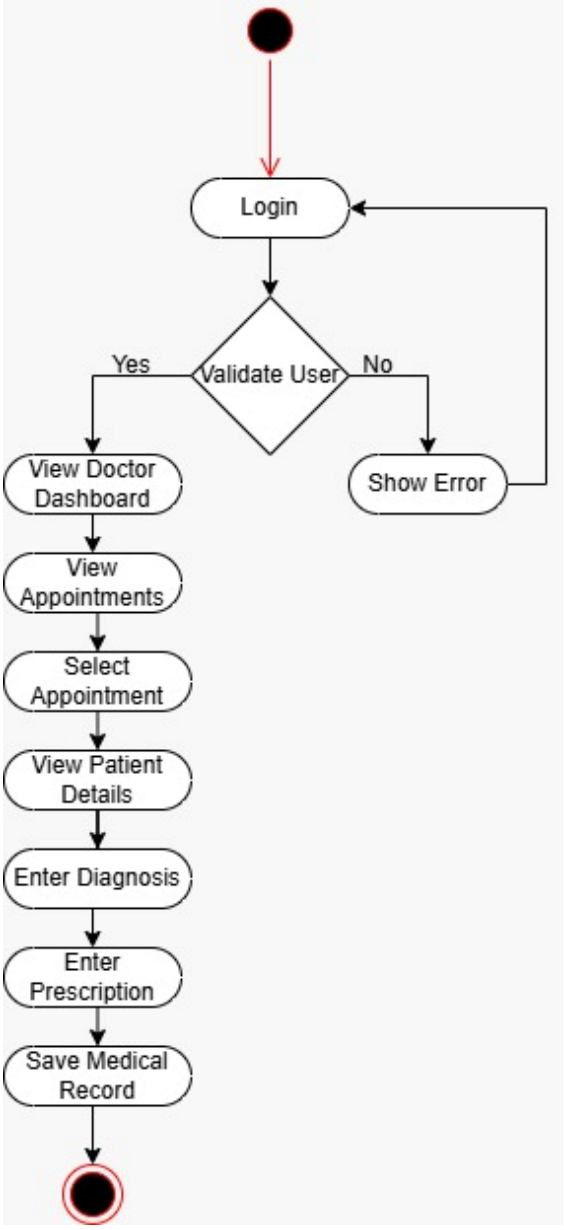


Figure 7 – Activity Diagram: Doctor Workflow

6.3 Manage Users

Clinic Admin logs in and opens User Management. They can Add a user (enter user data, assign role as Doctor or Receptionist, validate, save), Edit a user (search, select, update data, validate, update), or Delete a user (select, confirm deletion, delete). All users created are scoped to the Clinic Admin's clinic via ClinicId.

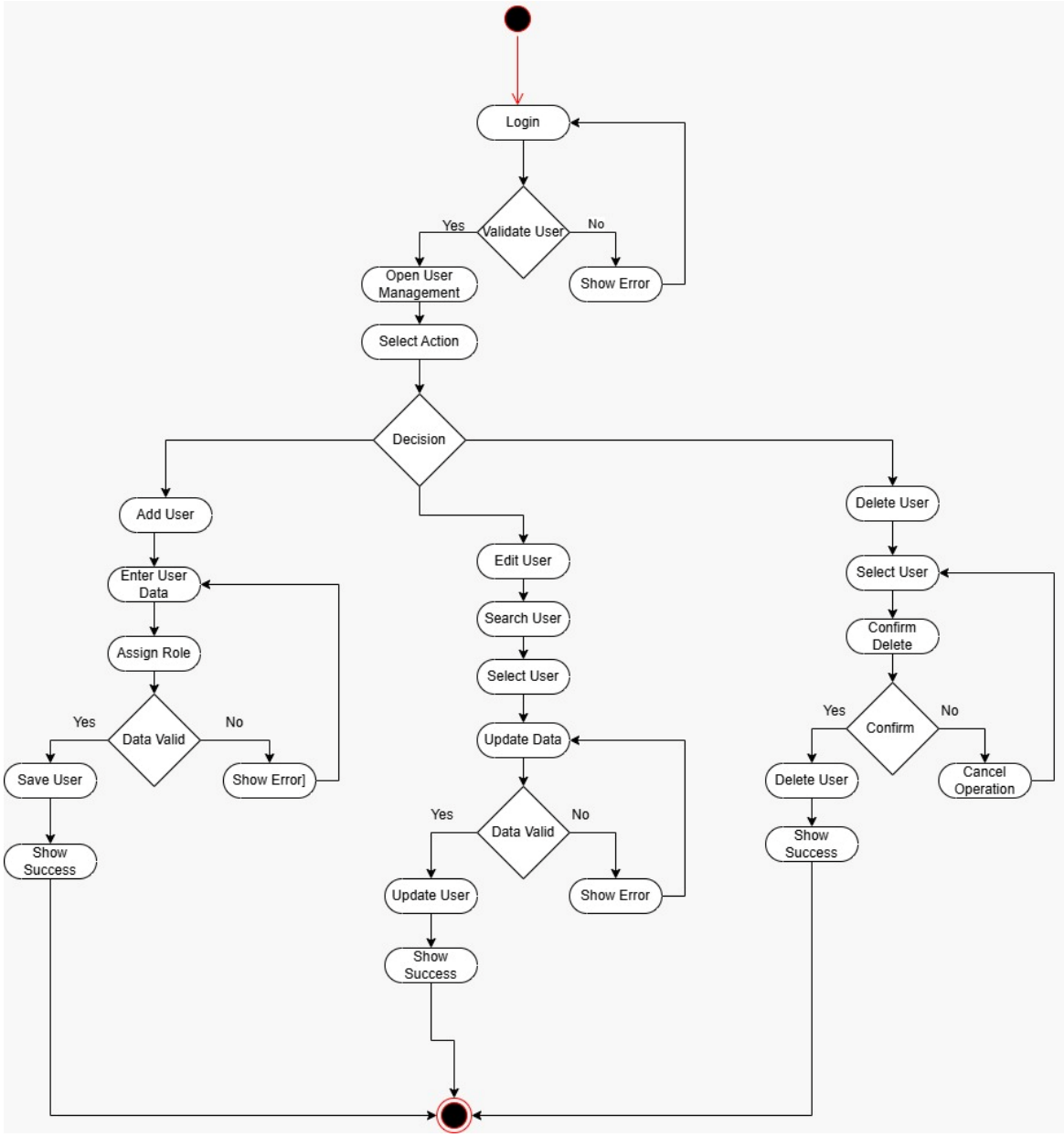


Figure 8 – Activity Diagram: Manage Users

6.4 Manage Patients

Receptionist logs in and opens Patient Management. They can Add a patient (enter data, validate, save), Edit a patient (search, select, update data, validate, update), or view existing patient records. The system validates all inputs before saving.

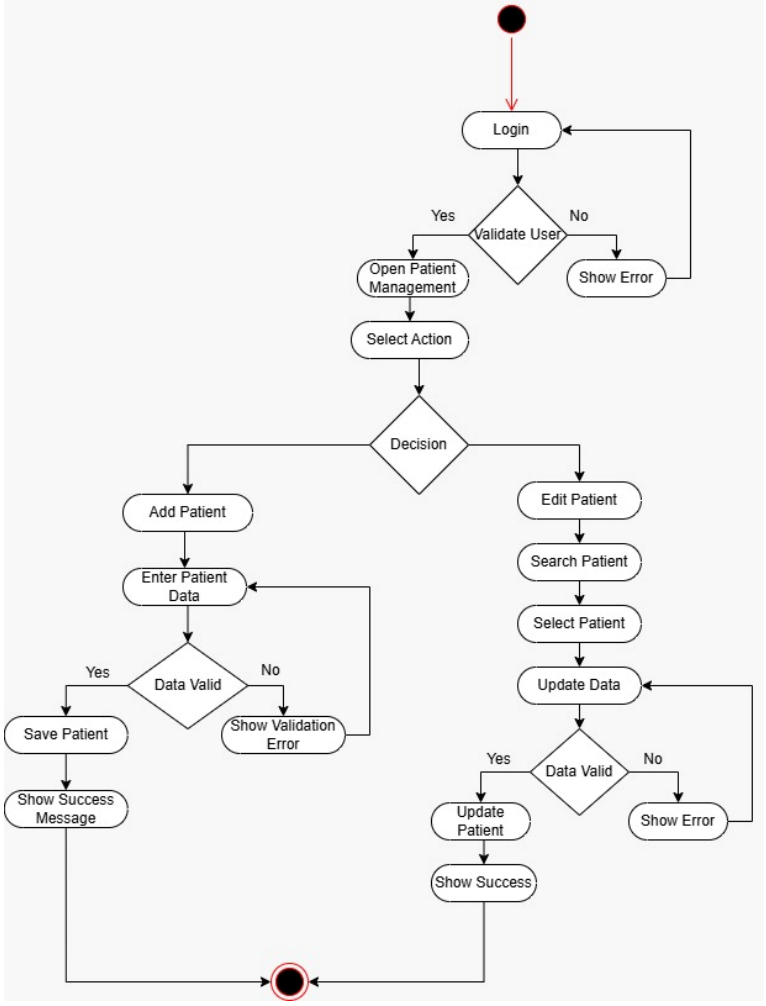


Figure 9 – Activity Diagram: Manage Patients

6.5 Book Appointment (Patient)

Patient logs in, views available doctors, selects a doctor, checks available time slots, selects a slot, confirms booking, saves the appointment, and receives a success message. If no slots are available, a notification is shown.

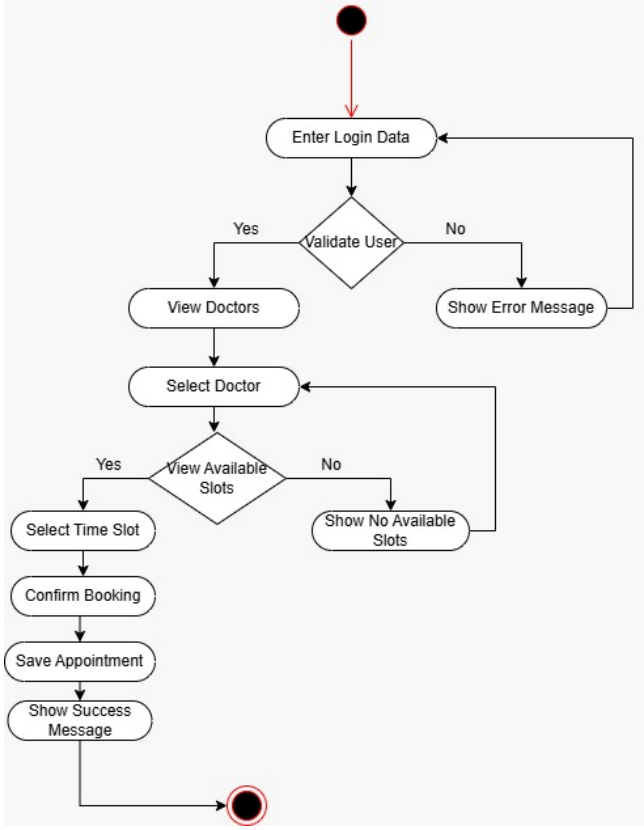


Figure 10 – Activity Diagram: Book Appointment (Patient)

6.6 Manage Appointments (Receptionist)

Receptionist can Add an appointment (select patient & doctor, choose time slot, save), Edit an appointment (select, modify date/time, check slot availability, update), or Cancel an appointment (select, confirm, delete).

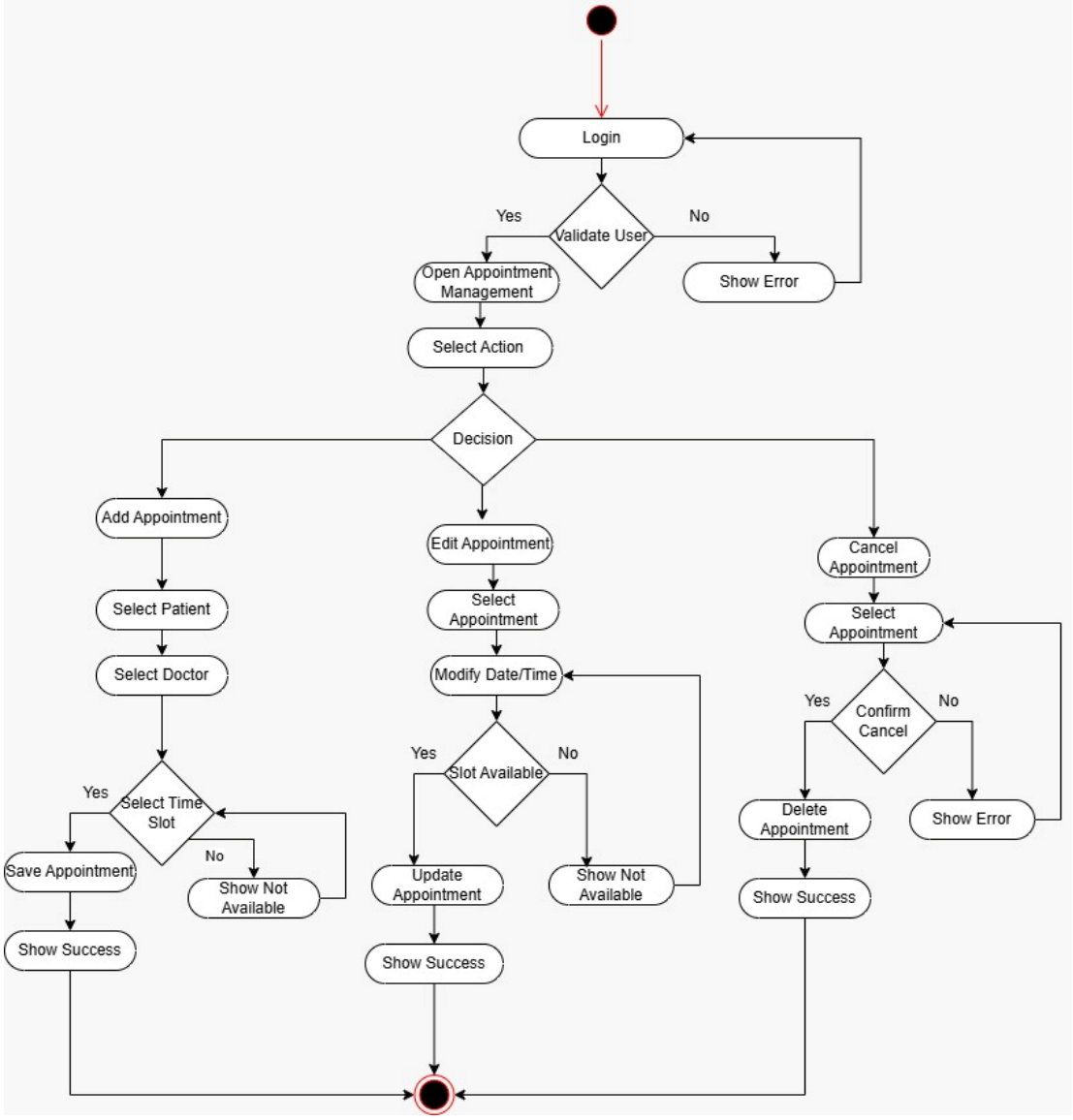


Figure 11 – Activity Diagram: Manage Appointments (Receptionist)

7. Sequence Diagrams

Sequence diagrams model the interactions between system components at runtime. Each diagram follows a consistent layered architecture: Frontend (Web/App) → Auth Controller → Tenant Service → Domain-Specific Controller (Patient / Appointment) → Business Logic Layer → Database (SQL Server). The Tenant Service resolves the ClinicId on every request to ensure multi-tenant data isolation.

7.1 Doctor: View Patient Records & Add Diagnosis

1. Authentication – Doctor enters credentials; Auth Controller validates against DB and returns User + Role + ClinicId; Frontend receives an Auth Token.

2. Resolve Tenant – Frontend retrieves ClinicId from Tenant Service for subsequent requests.

3. View Patient Records – Doctor selects a patient (GET /patients/{id}); Patient Records Controller resolves ClinicId, fetches patient data scoped to the clinic, and returns patient info.

4. Add Diagnosis – Doctor enters diagnosis & prescription (POST /patients/{id}/diagnosis); Business Logic Layer validates input; if valid, inserts diagnosis with ClinicId; success response returned. If invalid, failure response shown.

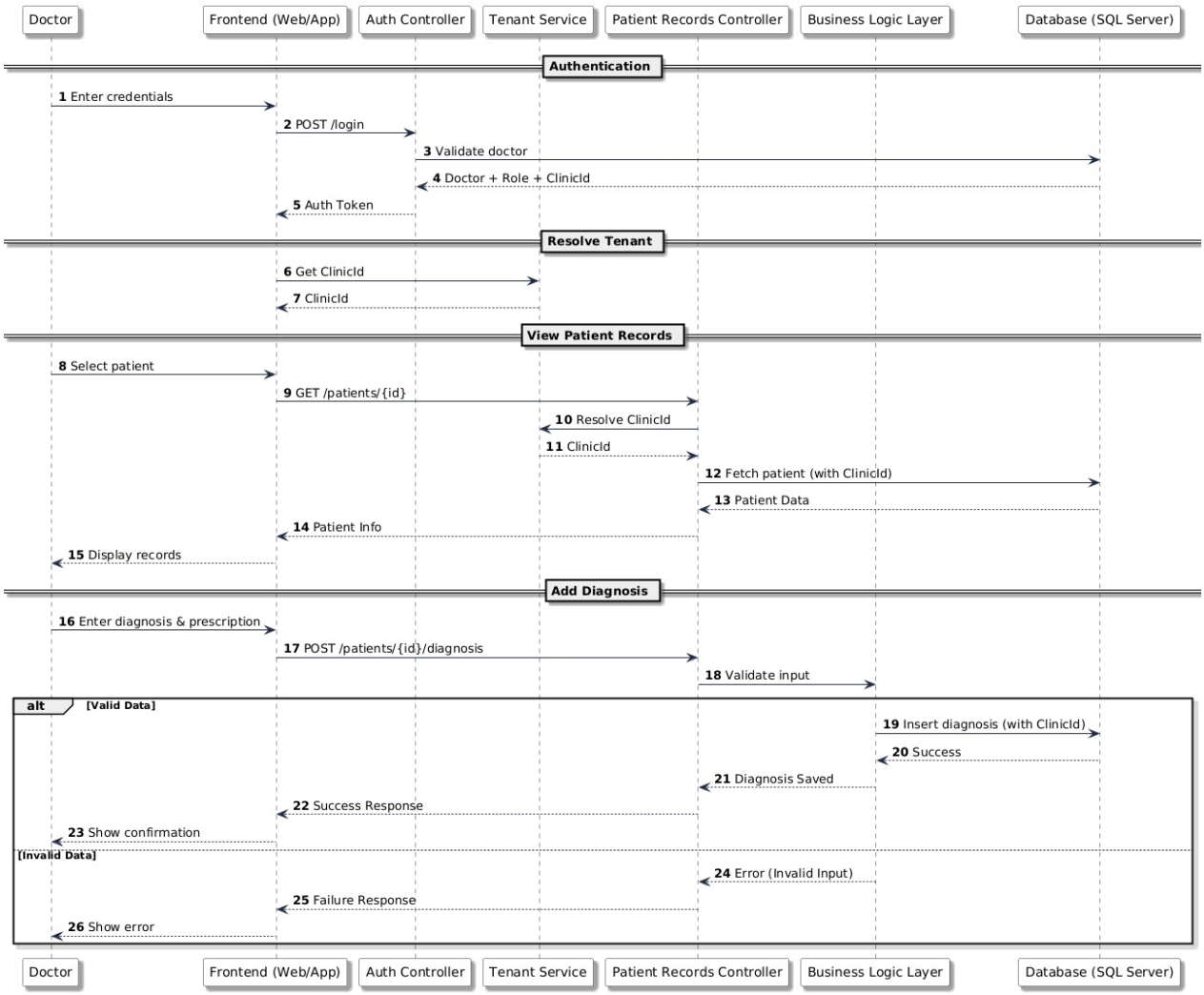


Figure 12 – Sequence Diagram: Doctor – View Patient Records & Add Diagnosis

7.2 Receptionist: Add/Update Patient | Patient: Book Appointment

Receptionist – Add/Update Patient: After authentication and tenant resolution, Receptionist sends POST /patients (or PUT /patients/{id}). Patient Controller resolves ClinicId, validates data, and inserts/updates the patient record with ClinicId in the database. Success or failure response is returned.

Patient – Book Appointment: After authentication and tenant resolution, Patient sends POST /appointments with selected doctor and time. Appointment Controller resolves ClinicId, validates the request, and checks doctor availability. If the slot is available, the appointment is inserted with ClinicId and a confirmation is returned. If the slot is taken, an error is returned.

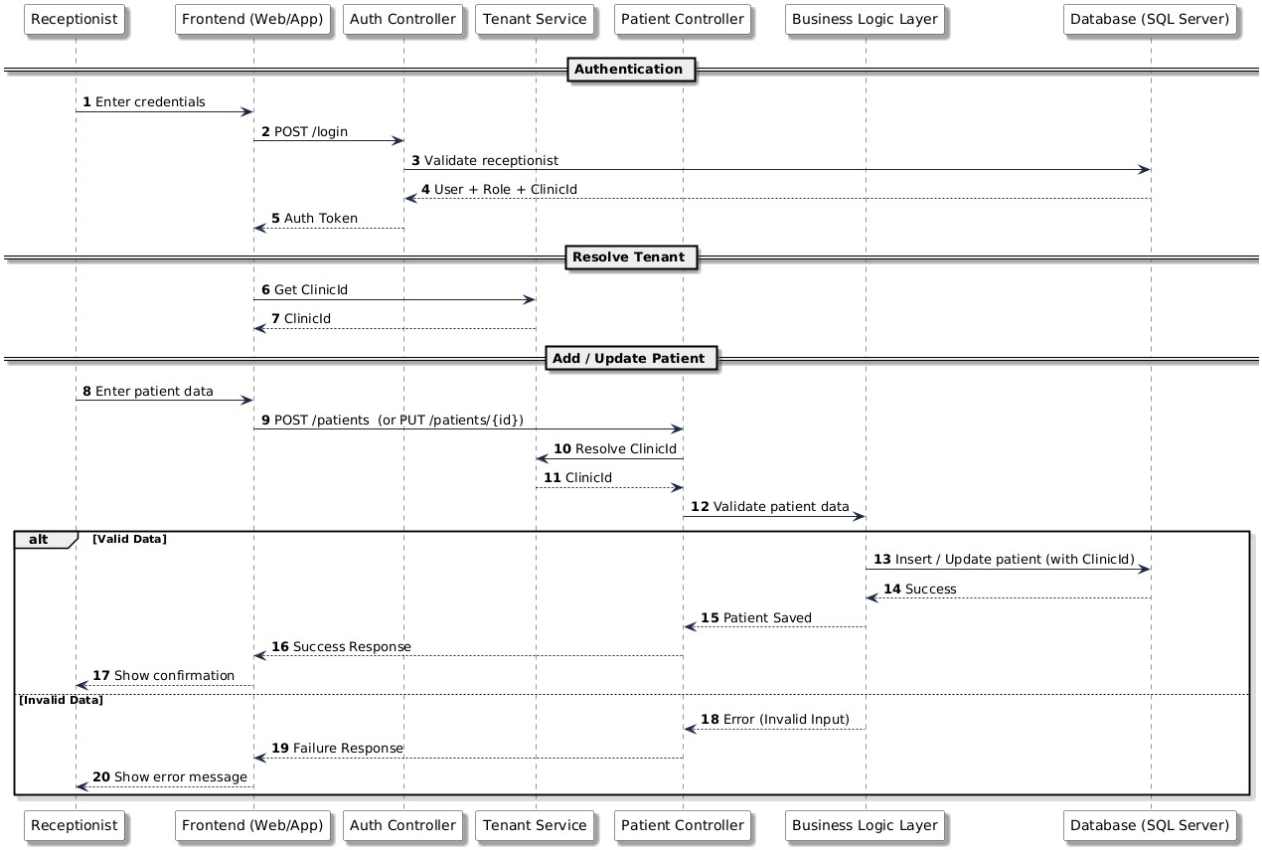
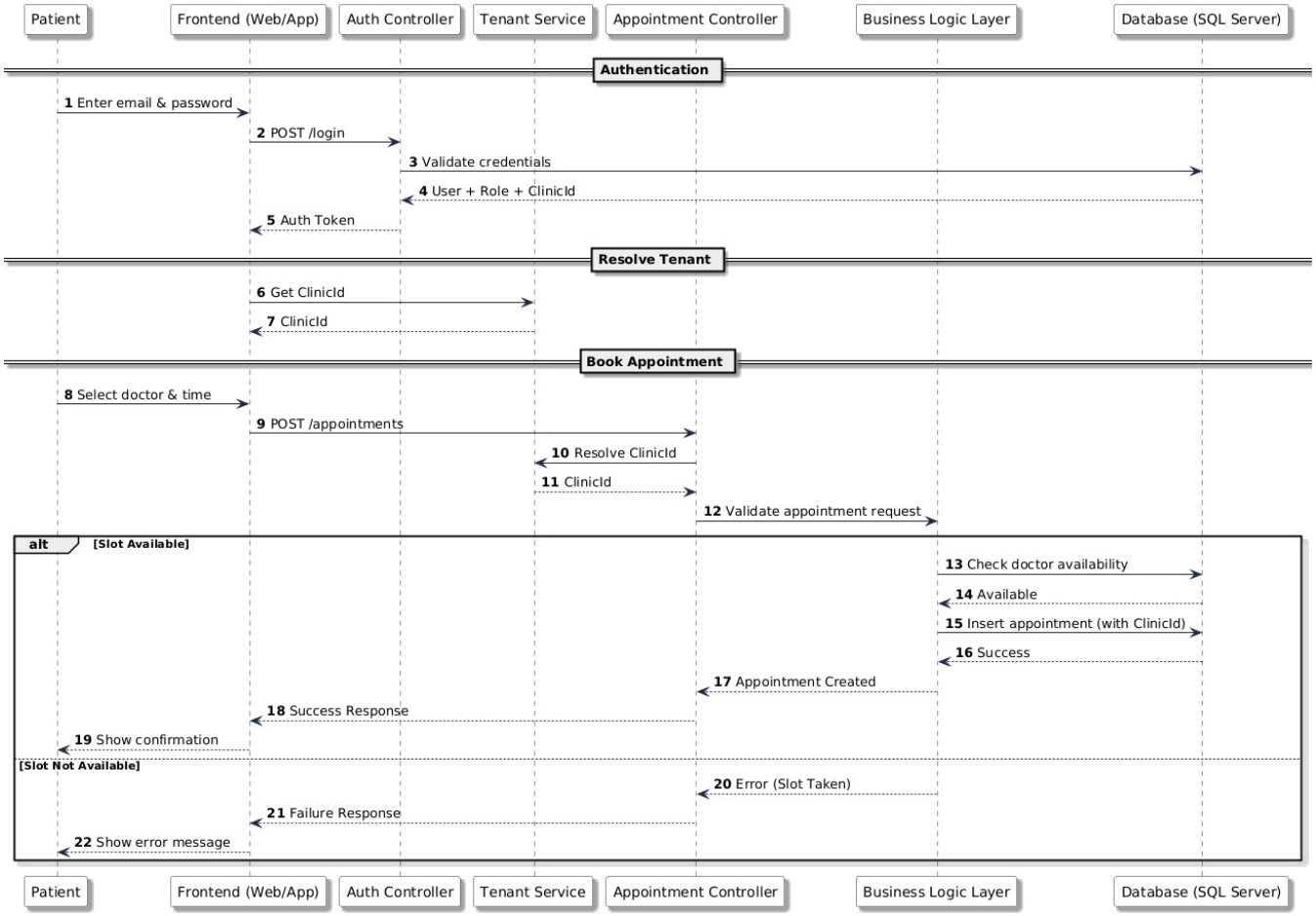


Figure 13 – Sequence Diagrams: Receptionist (Add/Update Patient) & Patient (Book Appointment)



8. Summary

The SmartClinic system is a well-structured multi-tenant platform. The design consistently enforces tenant isolation by propagating ClinicId through every layer – from authentication to data storage. The role-based access control model (SuperAdmin → ClinicAdmin → Receptionist / Doctor → Patient) provides a clear permission hierarchy.

All diagrams presented in this document are consistent with one another: entities in the class diagram correspond to tables in the ERD; processes in the DFD correspond to use cases in the use case diagram; and the sequence diagrams validate the runtime behaviour described in the activity diagrams.